IMSDA EVENTS / REVIEW & ENABLEMENT PACK

IMSDA Events: code structure and verification evidence

> Prepared October 21, 2026 · Repository baseline **5e367b7c**
> Current implementation, gated capabilities, accepted design and future plans are distinguished in the text. Repository evidence does not establish the serving production state.

Baseline: repository main at `5e367b7c4d1b3c173d37c0415e335be76170b300`. The latest application-source merge is `72a6cb214185cd7b74696f8913535f85330b690f`; subsequent merges added the design reference documents. This report uses read-only source inspection and actual GitHub Actions run/job/log summaries. No application code or tests were changed, and no new application test run was started for this report.

1. What the evidence establishes

The current main commit has a successful automated smoke run and successful application/backup-image builds. Its smoke job applied migrations to a clean CI PostgreSQL database, checked schema drift, seeded synthetic data, linted, generated/type-checked routes and ran Vitest. The recorded result is **1,015 test files passed; 13,504 tests passed and one skipped**, from 13,505 tests. The test step completed at 13:57 UTC on October 21.

The latest application PR also has a successful broader run: its configured real-database and built-application HTTP suites completed, along with successful optional browser badge and mobile checks. A repository comparison found 58 changed paths between that PR head and current main, all under `docs/`; application, script and workflow source did not change between those two revisions. This makes the earlier application-run evidence relevant while preserving its actual commit identity.

These are substantial engineering checks. They do not establish which image is currently serving production, which switches are enabled, whether a particular production import was approved, or whether every user journey meets accessibility requirements. “Not verified in this report” does not mean an activity was never performed. Existing rehearsals, decisions and import examples remain the evidence to consult; this report does not create replacement registers or rehearsals.

2. Architecture and source ownership

The accepted foundation is a **modular monolith**: one Next.js application and PostgreSQL database, with explicit business domains under `modules/`. It is not a collection of separately deployed microservices. Pages and route handlers orchestrate requests; domain services and repositories enforce business behavior and persistence. The database schema and committed migrations are part of the application release.

The checked package manifest uses Next.js 16.3.6, React 19.2.4, TypeScript, Prisma 6.19.3, Zod 4 and Vitest 4. CI selects Node.js 22. These are source-version facts rather than a dependency-vulnerability assessment. There are 70 domain directories, 1,015 Vitest test files and 218 committed SQL migration

files at the reviewed commit. File counts describe scale; they are not a code-coverage percentage or a quality score.

Layer	Main locations	Responsibility
User interface and HTTP	`app/`, `components/`, `proxy.ts`	Public/attendee/club/staff/system views, forms, route boundaries, response behavior and request flow.
Business domains	`modules/`	Authorization-aware workflows, validation, state transitions, pricing, capacity, assignments and reporting projections.
Shared infrastructure	`lib/`	Prisma access, environment validation, authenticated encryption, logging/redaction and shared validation utilities.
Persistence and reference data	`prisma/schema.prisma`, `prisma/migrations/`, seed/sync scripts	Relational constraints, release migrations, synthetic local data and versioned template/catalog synchronization.
Verification and operations	`tests/`, `scripts/`, `.github/workflows/`, `docs/`	Unit/route contracts, real-database/HTTP/browser checks, image builds and operator instructions.

The maintenance rule is to extend the owning module. A calendar page must not acquire its own registration-eligibility rules; a dashboard must not recalculate money differently from finance; an editable template library must not propagate changes directly into published registration versions. Shared visual controls are useful, but four workspaces still have distinct jobs and authority.

Domain map for code review

Business area	Primary module families	Important boundary
Accounts and access	`access`, `attendee-accounts`, `account-merge`, `people`, `guardian-authority`	Person identity, account credentials, sessions, household authority and event/club permissions remain separate concepts.
Registration and forms	`forms`, `registrations`, `attendee-types`, `consent`, `acknowledgments`	A form definition/version, registration party and individual attendee snapshot are different records.
Money and billing	`payments`, `promo-codes`, `scholarship-funds`, `billing-responsibility`, `invoices`, cancellation modules	Current money records and immutable invoice versions are the baseline; the general future ledger is not implied by a narrow meal/merchandise ledger.
Club ministry	`organizations`, `club-rosters`, `club-forms`, `club-registrations`, `club-reports`, `honors`, order/supply modules	Selected club/year, role differences, capacity, instructor scope and historical completion evidence must agree.
Event operations	`lodging`, `meals`, `issued-assets`, `program-assignments`, `checkin`, `event-locations`	Assignments, tickets, attendance, payment and readiness states are related but not interchangeable.
Communication and discovery	`communications`, `notifications`, `registration-form-notifications`, `calendar`, `events`	Outbox delivery, alert preferences, persisted notification history, public visibility and content publication have different ownership.

Business area	Primary module families	Important boundary
Restricted and operational evidence	`health-records`, `coordinator-health`, `audit`, `operations`, `system-admin`, `imports`	Protected detail, permitted projections, audit trails, manual readiness and measured operational health remain distinct.

Sources: foundation ADR, package manifest, schema.

3. The main request and data flows

Registration through operations

A public event/form is resolved through the public projection. The server loads the applicable published form version and validates the submitted registration and per-person answers. Pricing, capacity and assignment decisions must use server-owned state; the browser's displayed total or seat count is not authoritative. A successful transaction preserves the registration's form/version context, attendees, relevant reservations and history. Later edits and lifecycle changes go through the owning service rather than writing status columns directly.

Messages and downstream work are queued or triggered through existing service boundaries. Staff views then project the registration into financial, class, lodging, meal and check-in contexts. A waitlisted registration is not a confirmed seat; a check-in is not proof of payment; a screening status is not a medical approval. This separation is the practical reason to retain domain ownership when improving the interface.

The source contains explicit protections against a legacy import silently reactivating a cancelled/waitlisted registration and taking class seats without normal capacity checks. It is a useful example of why an import is a reviewed business operation, not simply a database upload.

Versioned forms and templates

`RegistrationForm`, `RegistrationFormVersion` and registration snapshots preserve which questions and rules applied. Existing save/publish paths serialize changes and return explicit conflict/busy results. First-publish-only testing is implemented; later form versions do not automatically require another builder test submission. Code tests and CI remain separate obligations.

Seeded form templates have immutable version records and a synchronized definition store. Forms copy definitions; they do not inherit live changes. The app-editable template library/source tracking in #1064/#1065 remains future work, so current source should not be described as already satisfying that full authoring contract. An ordinary registration form, whole-event template, saved question and fixed-key module bundle are also separate reuse units.

Sources: form repository, seeded-template service, form-builder owner #1062.

Money and organization billing

Current finance relies on existing Payment, Refund, RegistrationAdjustment, sponsor/scholarship and invoice records. Church-billed registrations are explicitly excluded from attendee online-balance totals and shown as church amounts; a group contact can be a different debtor from a church. A director submitting a club registration does not thereby owe the invoice personally.

Square has its own adapter/repository and provider-event handling. A provider acceptance, browser redirect or payment attempt must not be presented as final settlement without the applicable recorded result. Retries and webhook replay require idempotency, while refunds and financial adjustments retain

staff authority and audit. The general append-only-ledger transition in #117–#123 is later migration work; domain-specific order/meal ledgers do not prove that every balance reader already uses it.

Sources: finance projection, Square repository, invoice runbook, ledger owner #74.

Communication and background work

The communications boundary uses persisted outbox records, provider events, retry/backoff and scheduled sweeping. The sweep also dispatches due batches that a person has already approved. Operational queue health includes counts and oldest-due age; an application process answering HTTP alone is not enough to declare email healthy.

The notification catalog/switches, live action counts, and future persisted administrator feed are separate pieces. #1033 still owns feed/read/unread/retention work; central Email settings #1043 are delivered. SES accepting SMTP delivery is not the same as inbox delivery. The SES/SNS bounce/complaint integration in #863 must not be counted as implemented merely because equivalent Resend webhook infrastructure exists.

Sources: outbox sweep, notification catalog, delivery follow-up #863.

4. Authentication, roles and protected data

One sign-in does not mean one unrestricted session

The accepted account model links `User` and `AttendeeAccount` 1:1. It retains their underlying credentials and session boundaries. Unified password sign-in evaluates both credential sides and only staff-grade authentication can issue staff authority. A valid attendee password or attendee email link does not become a staff session because the accounts are linked. Staff passkeys/second factor, staff activation, verified attendee linkage and switch-off fallback have their own existing services.

The feature is controlled by the persisted unified-sign-in switch, off by default. This report verified code and CI, not its production value. Profile/workspace links show only authorized contexts. The accepted administrator menu and distinct attendee/director/coordinator layouts remain presentation contracts rather than a new permission engine.

Event and club authority

`requireEventMembership`, `requirePermission` and `effectivePermissions` are central staff/event checks. Current SYSTEM_ADMIN is permitted into events without an EventMembership and receives the event permission set; a UI redesign must not silently reverse that rule. Specialized domains can impose further guards. Kitchen-only staff deliberately have `MANAGE_KITCHEN_COUNTS` without general `VIEW_EVENT` and must not gain registration access through a new dashboard link.

Club access is separately selected by club/year and actual grant: deputy, registrar and reporter are not equivalent to director. A public management link, multiple people on a registration, common email, Person link or household association must not be treated as blanket guardian/medical authority. Coordinator scope must follow the current owning query; future ministry filtering in #843 is not present merely because its direction was accepted.

Encryption and health scope

`lib/secret-box.ts` supplies authenticated AES-256-GCM with fresh nonces and purpose-separated HKDF keys. It supports the protected-key file configuration. This establishes a primitive and actual uses,

not proof that every legacy field or export is encrypted. Roster birth dates, the Pathfinder Health Record and the separate coordinator event sheet have different datasets and policy addenda.

Current health-record paths include a feature gate, role/purpose checks, audit, one-time acknowledgment, registrar creation-only behavior, event access-window checks, sealed snapshots/holds and administrator break-glass enforcement. A system administrator's own applicable grant is required by the Health Record repository before protected decryption; act-as does not bypass that requirement. The separate coordinator sheet reads existing registration/club-form data under its own window and permission contract. The overall ADR and each addendum must be read at their own accepted/interim/proposed status.

The source does not establish a complete operational key-version/rotation program or prove a real key-backup restore. Existing ADR prerequisites, operator evidence and human decisions remain authoritative. No real protected values were accessed for this report, and this is not a HIPAA or security-conformance certification.

Sources: authorization, permission map, unified sign-in, ADR 0013, encryption primitive, protected-record ADR/addenda.

5. Imports, migrations and operational release

Imports already have parsers, reviewed rows, source identifiers/snapshots, proposed actions, conflicts and apply outcomes. Club imports, eAdventist organization imports, WR26 staging/bundles and health imports have different owners and allowlists. An import profile must not confer authority or revive capacity-consuming records by directly replacing their status.

Existing JSON reference data, CSV fixtures and example PDFs remain useful input/output evidence. Keep using them rather than recreating another set for this documentation pack: reference imports, public CSV fixtures, registration PDF samples. Synthetic examples demonstrate formats; a production reconciliation remains tied to its reviewed file/version and approval.

CI deploys the 218 committed migrations into its throwaway database and compares the resulting migration history to the Prisma schema via a shadow database. That is useful evidence against drift on a clean installation. It does not prove the exact state or upgrade path of an existing production database, nor authorize `migrate dev` against it. Production release/import/backup actions follow the existing operator runbooks.

Docker image CI publishes application and backup images tagged with the commit SHA. The immutable SHA tag is the release identity to compare with the running service; a successful publish does not show that the server pulled or started it. Runtime credentials, trusted-proxy/rate-limit setup, storage mounts, scheduled workers, email delivery and external provider configuration require their applicable environment evidence.

Sources: import repository, deployment guide, backups, server checklist.

6. What runs automatically, and when

The canonical local command is `npm run verify`: lint, Next-generated route types/TypeScript, Vitest and production build. Additional scripts exercise real database constraints, concurrency, imports and HTTP paths. They are not all included in that single command.

Check layer	Current configured trigger/scope	Interpretation
CI `main-smoke`	Push to main	Clean migrations/drift/seed, lint, typecheck and Vitest. No production build or extended domain/HTTP suites in this job.
CI `verify`	PR, weekly schedule or manual	Installs/migrates/drift-checks/seeds/syncs and runs `verify`. Database/HTTP extensions run for non-visual changes and scheduled/manual runs; visual/docs-only PRs skip those extensions.
Mobile layout	Selected helper/workflow paths or `mobile-check` label; weekly/manual	PR quick run is 360/1024 with screenshots disabled. Weekly/manual full run uses 360/390/768/1024 with screenshots. Job is optional: inspect actual check step and report.
Badge print	Badge/print dependencies, weekly/manual	Built-app Chromium/PDF check, optional job. Physical media/printer alignment and scan reliability remain separate.
Docker image	Push main/manual	Application and backup image build/push. Publication is not deployment or an end-user test.

The mobile audit checks overflow, table/card behavior, target sizes, dialog/sticky-bar geometry, headings, labels, overlap, some contrast rules and related layout conditions. It is a valuable deterministic screen check, not a full screen-reader, keyboard, cognitive-usability or WCAG assessment. Browser jobs use synthetic local/CI data. Their scripts contain guards that refuse a remote/production target.

Sources: CI workflow, mobile workflow, responsive rules.

7. Observed verification ledger

All times in this section are UTC on October 21, 2026. Run links and detailed step metadata are also recorded in `evidence/technical-checks.json` supplied with this pack.

Observed evidence	Actual outcome	Practical limit
Current main CI, run 39609956369, SHA `5e367b7c`, created 13:39, completed 13:57	`main-smoke` successful. Migrations, drift check, seed, lint, typecheck and Vitest steps successful. 13,504 tests passed, one skipped, 1,015 files passed; tests took 225.99 seconds.	`verify` job skipped on push by design. No new full-domain/HTTP/browser run at this exact merge SHA was observed.
Current main images, run 39609956750, SHA `5e367b7c`, 13:39–13:54	Both application and backup-image build/push steps successful.	Does not establish running deployment, real provider credentials or database upgrade state.
Application-source merge smoke, run 39522715128, SHA `72a6cb21`, 00:40–00:49	Main-smoke steps successful.	Historical merge-run evidence, distinct from current documentation merge.

Observed evidence	Actual outcome	Practical limit
Latest application PR full gate, run 39520473876, SHA `9879edba`, created 00:18	`verify`, all configured extended real-database checks, built-app startup and seven public/operations HTTP checks successful. Vitest summary: 13,504 passed, one skipped.	This is the PR head, not current main's commit. A git comparison found application/script/workflow content unchanged through current main; only 58 documentation paths differ.
Application PR mobile, run 39520478915, SHA `9879edba`, actual check completed 00:34	Actual “Check phone layouts” step successful: 320 page/width combinations, 301 dialogs, 1,871 boundary-checked form fields and 56 dialog date fields; zero accepted findings and zero page script errors reported.	Quick widths 360/1024 only; screenshots disabled. The artifact is a small report, despite its `mobile-layout-screenshots` name. Not an all-width or WCAG certification.
Application PR badge check, run 39520473962, SHA `9879edba`	Actual “Check name badge printing” step successful; `badge-print-output` artifact present and unexpired when inspected.	Automated print rendering, not physical printer/media/device verification.
Design-reference PR 1119 CI, SHA `b7b39e95`, and PR 1110 CI, SHA `696743c3`	`verify` including build successful; clean migrations/drift/seed and sync steps successful.	Scope classification skipped all extended real-database/HTTP suites. Their separate mobile-label runs were skipped, not passed browser audits.

The one skipped current-main Vitest case is in `tests/backup-scripts.test.ts`: “as root, drops to postgres with gosu, and refuses without gosu or su-exec.” Its `skipIf` condition runs it only as root, unlike the normal GitHub-hosted runner user. This is an explicit environment-specific coverage limit, not evidence that every backup scenario ran there. Negative-path tests can also intentionally log redacted errors; the final assertion and step conclusions, rather than the mere presence of an “error” line, determine a test result.

8. Existing checks and evidence that should keep their original scope

The earlier public-journey review reports 137 passing existing tests in eight files against application source `72a6cb21`. Its dependency/client setup details are documented. The corrected role-design pack reports 16 static screen/width cases, 11 role/state checks and three gallery widths. These validate the reviewed contracts or reference mockups; they are not fresh claims that proposed layouts are already implemented.

Older production-readiness documents contain valid historical work, including synthetic backup/restore and deployment-related checks, but they also quote older test/migration counts and dated judgments. Preserve their date/commit rather than carrying their headline “ready” status forward indiscriminately. The user already has rehearsal plans, decision material and import examples; attach the applicable existing evidence to the release being reviewed rather than create another competing checklist.

The open load-sensitive test issue #917 remains relevant. A successful run is evidence of that run, not proof that intermittent cold-import, hashing, PDF or timing-race failures have been eliminated. The issue's repeated-run acceptance should be assessed when that bounded work is performed, without skipping or weakening assertions.

References: public-journey validation, role reference validation, readiness progress, dated WR26 go-live review.

9. Using this report in a code or release review

For a future bounded code change, first select the existing owner from roadmap #98, inspect current source and use the issue's residual acceptance contract. Keep the accepted role/menu direction. Review the actual changed domain, authorization entry points, database constraints, retries/concurrency, version/history behavior and public/export projections. Run proportional checks, then verify the required CI steps on the resulting commit rather than assuming a green label means every suite ran.

For an operational release, pair that code evidence with the existing environment-specific records: serving image/release identity, migration state, reviewed import apply, worker/backup status, real key restore evidence, payment/provider setup, physical print/scan checks and the applicable human sign-offs. This report neither asserts those are absent nor substitutes for them. Production account linking, medical handling, refunds, bulk messages, pricing/capacity changes and deployment remain governed by the repository's existing human-action rules.

The main documentation improvement is traceability: each claim should name **what was checked, on which source/environment, by which recorded run, with what outcome and what limit**. That makes both current strengths and real remaining work easier to assess without reopening completed work or overpromising future features.